

OBSERVACIONES DE LA PRÁCTICA

Estudiante 1 Cod XXXX

Estudiante 2 Cod XXXX

Estudiante 3 Cod XXXX

Ambientes de pruebas

	Máquina 1	Máquina 2	Máquina 3
Procesadores	[Completar: Panel de control > Sistema > Procesador]		
Memoria RAM (GB)	[Completar: Panel de control > Sistema > RAM instalada]		
Sistema Operativo	Windows		

Tabla 1. Especificaciones de las máquinas para ejecutar las pruebas de rendimiento.

Máquina 1

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%	0.080	0.095	0.004
5.00%	0.408	0.464	0.002
10.00%	0.661	1.095	0.001
20.00%	1.419	2.883	0.007
30.00%	1.968	3.608	0.002
50.00%	4.409	7.411	0.002
80.00%	5.503	18.440	0.002
100.00%	7.607	24.346	0.003

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Linked List)
0.50%	0.035	0.053	0.003

5.00%	0.347	0.452	0.001
10.00%	0.683	1.012	0.001
20.00%	2.487	1.999	0.031
30.00%	1.936	2.501	0.001
50.00%	4.615	4.311	0.002
80.00%	7.501	6.771	0.003
100.00%	7.438	9.875	0.002

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%	0.244	0.130	0.019
5.00%	0.918	1.234	0.002
10.00%	1.199	1.048	0.002
20.00%	3.376	2.210	0.002
30.00%	3.536	4.249	0.009
50.00%	6.514	7.585	0.003
80.00%	11.359	9.381	0.003
100.00%	19.836	13.144	0.003

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%	0.070	0.495	0.006
5.00%	0.689	7.103	0.003
10.00%	0.866	23.773	0.002
20.00%	2.783	100.866	0.003
30.00%	2.956	260.109	0.003
50.00%	6.386	744.605	0.005
80.00%	9.516	1814.438	0.004
100.00%	28.190	2836.770	0.005

Máquina 2

Resultados para Queue con Array List

Porcentaje	enqueue (Array	dequeue (Array	peek (Array
------------	----------------	----------------	-------------

de la muestra	List)	List)	List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Máquina 3

Resultados para Queue con Array List

Porcentaje de la muestra	enqueue (Array List)	dequeue (Array List)	peek (Array List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Stack con Array List

Porcentaje de la muestra	push (Array List)	pop (Array List)	top(Array List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Queue con Linked List

Porcentaje de la muestra	enqueue (Linked List)	dequeue (Linked List)	peek Linked List
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Resultados para Stack con Linked List

Porcentaje de la muestra	push (Linked List)	pop (Linked List)	top(Linked List)
0.50%			
5.00%			
10.00%			
20.00%			
30.00%			
50.00%			
80.00%			
100.00%			

Preguntas de análisis

1. ¿Se observan diferencias significativas entre las implementaciones con ArrayList y LinkedList para las funciones de Queue y Stack? ¿Cuál es más eficiente en cada operación? ¿Por qué una implementación es más rápida en ciertos casos?

Sí, y son marcadas en varias operaciones. Con la muestra completa (10.000 libros, 100%): en la Cola, enqueue es más rápido en Array List (7.607 ms) que en Linked List (19.836 ms), mientras que dequeue es más rápido en Linked List (13.144 ms) que en Array List (24.346 ms); peek es prácticamente idéntico en ambas (0.003 ms), como corresponde a una operación $O(1)$ en las dos implementaciones. En la Pila, push es más rápido en Array List (7.438 ms) que en Linked List (28.190 ms), y la diferencia se dispara en pop: Array List toma 9.875 ms frente a 2836.770 ms de Linked List, casi 287 veces más lento. La razón está en la implementación: add_last en Array List es un append de Python ($O(1)$ amortizado, memoria contigua) y en Linked List arma un nodo y actualiza el puntero last (también $O(1)$ pero con más overhead

por objeto). `remove_first` en Linked List solo mueve el puntero `first` ($O(1)$ real), mientras que en Array List cada `remove_first` desplaza todos los elementos restantes ($O(n)$ por operación). Y `remove_last` en Linked List es el punto débil de esta implementación: al no guardarse un puntero al penúltimo nodo, hay que recorrer la lista completa para encontrarlo, así que cada `pop` es $O(n)$ y el ciclo de prueba (que hace n pops) termina siendo $O(n^2)$; en Array List, `pop` al final es simplemente `list.pop(-1)`, $O(1)$, por lo que el ciclo completo es $O(n)$.

2. ¿Cuándo es preferible usar ArrayList o LinkedList? Si insertamos y eliminamos con frecuencia, ¿qué estructura conviene más? Si accedemos aleatoriamente a elementos, ¿cuál es más eficiente?

Depende del patrón de acceso. Si se inserta y elimina con frecuencia por el frente (como en una cola), Linked List conviene más, porque `add_last` y `remove_first` son $O(1)$ reales sin desplazar memoria; en Array List cada `remove_first` es $O(n)$. Si en cambio se accede aleatoriamente a elementos por posición (`get_element`), Array List es claramente más eficiente: indexar un arreglo es $O(1)$, mientras que en Linked List hay que recorrer nodo por nodo desde el inicio, $O(n)$. Para operaciones concentradas al final de la estructura (como una pila con `push/pop`), Array List es la mejor opción con esta implementación puntual, porque `remove_last` en la Linked List usada aquí no mantiene un puntero al penúltimo nodo y por eso es $O(n)$ en vez de $O(1)$.

3. Durante la ejecución de las pruebas ¿Se presentan anomalías en los tiempos de ejecución que no se explican con la teoría?

La anomalía más notoria está en `dequeue`: en teoría, el ciclo de prueba hace `remove_first` n veces, así que Array List debería ser $O(n^2)$ total (cada `remove_first` desplaza hasta n elementos) frente a Linked List, que es $O(n)$ total (cada `remove_first` es $O(1)$). Con $n = 10.000$ se esperaría una diferencia mucho más grande que la observada (24.346 ms vs 13.144 ms, solo 1.85x), muy por debajo de la brecha de ~287x que sí se ve en `pop` de la pila (donde la comparación asintótica es la misma, $O(n^2)$ contra $O(n)$). La explicación no contradice la teoría, la matiza: `list.pop(0)` de Python está implementado en C como un `memmove` contiguo muy optimizado, con una constante de tiempo mucho menor que recorrer nodos de un diccionario de Python (el nodo de Linked List es un dict con punteros). A este tamaño de muestra, la ventaja asintótica de Linked List todavía no alcanza a compensar su mayor costo por paso; con muestras más grandes se esperaría que la curva cuadrática de Array List termine dominando con más claridad.

4. Complete la siguiente tabla de acuerdo con qué operación es más eficiente en cada implementación (marque con una x la que es más eficiente). Adicionalmente, explique si este comportamiento es acorde con lo enunciado en la teoría. Justifique las respuestas.

		Array List	Linked List	Justificación
	Enqueue()	X		Array List es más rápida: <code>append</code> al final es $O(1)$ amortizado sobre memoria contigua, con menor overhead que crear y enlazar un nodo.

QUEUE	Dequeue()		X	Linked List es más rápida en teoría ($O(1)$ real, solo mueve el puntero first) y también en la medición a $n=10.000$, aunque la brecha es menor de lo esperado por el bajo costo del memmove de Python en Array List.
	Peek()	X	X	Empate técnico: ambas son $O(1)$ (indexación directa en Array List, puntero first en Linked List); el tiempo medido es prácticamente idéntico (0.003 ms).
STACK	Push()	X		Igual razón que enqueue: append en Array List es más barato que crear y enlazar un nodo nuevo.
	Pop()	X		Diferencia enorme (9.875 ms vs 2836.770 ms, ~287x): pop en Array List es $O(1)$ (list.pop(-1)), pero remove_last en esta Linked List es $O(n)$ porque no guarda puntero al penúltimo nodo, así que debe recorrer toda la lista en cada pop; el ciclo completo de prueba es $O(n^2)$.
	Top()	X	X	Empate técnico: ambas mantienen un puntero al último elemento ($O(1)$); los tiempos medidos son igualmente bajos (0.002-0.005 ms).